Hospitality Application Deployment & DevOps Pipeline

Course Name: DevOps Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Abhishek Gupta	EN22EL301004
2	Tanish Israni	EN22IT301112
3	Sourabh Patel	EN22IT301106
4	Samarth	EN22IT301087
5	Yash Dashore	EN22CS3011100
6	Ramakant Singh	EN22IT301077

Group Name: Group 10 D12

Project Number: DO-48

Industry Mentor Name: Mr. Vaibhav Sir

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2026

Introduction –

The hospitality industry relies heavily on digital applications to manage services such as bookings, customer interactions, and operational activities. Modern hospitality systems must operate continuously with minimal downtime while adapting quickly to changing business requirements. Traditional deployment methods based on manual configuration often result in inconsistent environments, delayed releases, and increased chances of system failure. To address these challenges, organizations are increasingly adopting DevOps practices that promote automation, scalability, and reliable software delivery.

The **Hospitality App DevOps Project** focuses on implementing an automated deployment pipeline for a hospitality management application using modern cloud-native technologies. The project integrates GitHub for version control, Docker for application containerization, and Kubernetes for container orchestration. Containerization ensures that the application runs consistently across different environments by packaging the application along with its dependencies into a single portable unit, thereby eliminating environment-related issues.

A key feature of this project is the implementation of a **Continuous Integration and Continuous Deployment (CI/CD)** workflow using GitHub Actions, combined with Infrastructure as Code (IaC) using Terraform. Every code update automatically triggers the build, container image creation, and deployment process to a Kubernetes cluster, enabling faster and reliable releases with minimal manual intervention.

Overall, the project demonstrates how DevOps principles improve deployment efficiency, system reliability, and scalability, making the hospitality application suitable for modern production environments.

Problem Statement & Objectives

1. **Problem Statement:**

   Traditional deployment approaches for hospitality management applications rely on manual processes, resulting in inconsistent runtime environments across development, staging, and production. These methods lead to slow release cycles, risk of configuration drift, human error during deployment, and inability to scale rapidly during peak demand periods such as festival seasons or large events. There is a critical need for an automated, containerized, and orchestrated infrastructure that ensures zero downtime, high availability, and repeatable deployments without manual intervention.

2. **Project Objectives:** To automate the end-to-end CI/CD pipeline for the hospitality management web application using GitHub Actions, enabling continuous integration and deployment on every code commit to the main branch.
    - To containerize the FastAPI-based hospitality application using Docker, ensuring consistent and portable execution across all environments.
    - To implement Infrastructure as Code (IaC) using Terraform for provisioning and managing AWS EKS cluster resources in a reproducible and version-controlled manner.
    - To ensure application reliability and zero downtime deployments through Kubernetes RollingUpdate strategy, readiness probes, and liveness health checks exposed via the /healthz endpoint.

3. **Scope of the Project:**
   This project covers the complete end-to-end DevOps lifecycle of the hospitality management application, beginning from source code management in GitHub through to production deployment on AWS Elastic Kubernetes Service (EKS). The scope includes containerization of a FastAPI Python application using Docker, definition of Kubernetes manifests for deployment and service resources, automation of build and deployment workflows via GitHub Actions CI/CD pipelines, and cloud infrastructure provisioning through Terraform. The project demonstrates production-grade practices including zero-downtime rolling updates, horizontal scalability with two replicas, load balancing via a Kubernetes LoadBalancer service, and automated health monitoring using HTTP readiness and liveness probes.

Proposed Solution

- The proposed solution implements a comprehensive DevOps ecosystem centred around GitHub Actions for CI/CD automation, Docker for application containerization, Kubernetes (AWS EKS) for container orchestration, and Terraform for Infrastructure as Code. The hospitality management application is developed using FastAPI (Python), packaged as a Docker image, and deployed as a Kubernetes workload with a RollingUpdate strategy that guarantees zero downtime during each release cycle. A LoadBalancer service exposes the application to end users, while two replicas ensure continuous availability even during pod replacement cycles.

Key Features

- ***Automated CI/CD Pipeline:*** *Every push to the main branch of the GitHub repository automatically triggers a GitHub Actions workflow that builds the Docker image, tags it with the commit SHA for traceability, pushes it to Docker Hub, and applies the updated image to the Kubernetes deployment via a rolling update.*

- ***Containerization with Docker:*** *The FastAPI application is packaged into a Docker image along with all its runtime dependencies, eliminating environment-specific configuration issues and enabling consistent execution across local, staging, and production environments.*
- ***Infrastructure as Code (Terraform):*** *AWS EKS cluster infrastructure is defined and provisioned using Terraform (main.tf), ensuring that cloud resources are version-controlled, reproducible, and auditable, following modern Infrastructure as Code principles.*
- ***Scalability and High Availability:*** *The Kubernetes deployment maintains two active replicas of the application at all times. A Kubernetes LoadBalancer service distributes incoming traffic across all healthy pods, ensuring high availability during peak booking periods and enabling horizontal scaling when demand increases.*
- ***Health Monitoring:*** *The application exposes a dedicated /healthz HTTP endpoint which Kubernetes uses for both readiness and liveness probes. Kubernetes periodically queries this endpoint to verify pod health, automatically restarting or removing pods that fail health checks and ensuring only healthy instances serve traffic.*

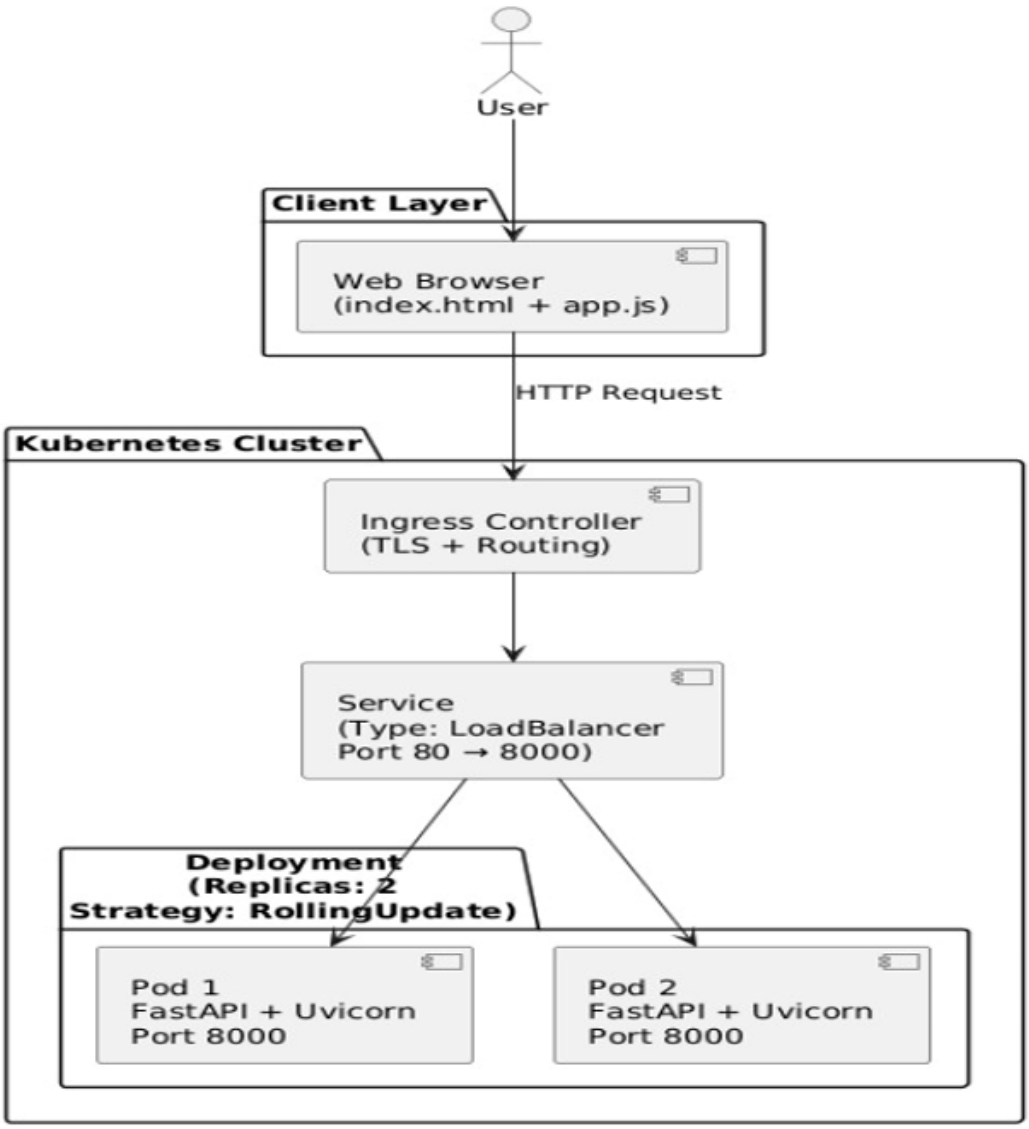
Overall Architecture / Workflow

1. **Code Management:** Developers write and commit FastAPI application code to the GitHub repository. All source files, Dockerfile, Kubernetes manifests (deployment.yaml and service.yaml), and Terraform configuration (main.tf) are version-controlled within the same repository, forming a single source of truth for the entire application and its infrastructure.
2. **Continuous Integration:** A push event to the main branch automatically triggers the GitHub Actions CI workflow. The pipeline checks out the source code, authenticates with Docker Hub using stored GitHub Secrets, builds the Docker image, and tags it using the unique Git commit SHA (github.sha) to maintain full image traceability across every release.
3. **Container Registry:** Upon a successful build, the Docker image tagged with the commit SHA is pushed to Docker Hub, serving as the centralized container registry. This makes the versioned image available for Kubernetes to pull during the deployment stage and provides a historical record of all application releases.

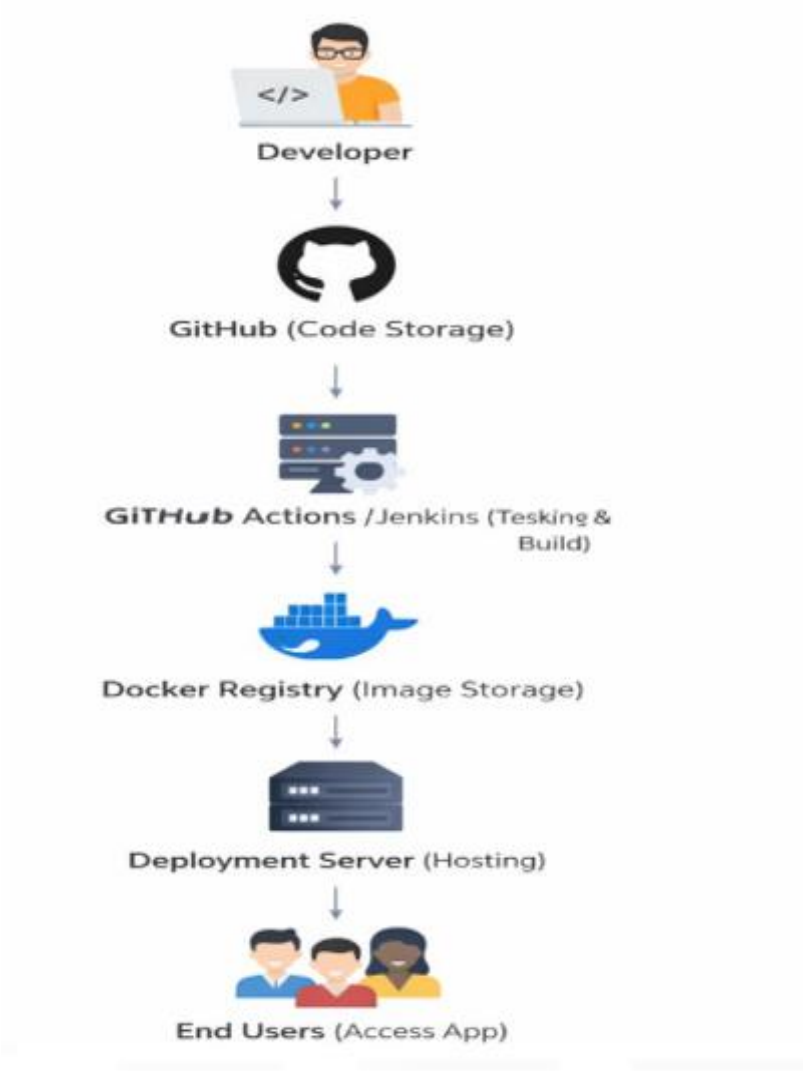
4. **Continuous Deployment:** The CD stage of the pipeline uses kubectl to update the image field of the Kubernetes deployment resource on the AWS EKS cluster with the newly pushed image tag. Kubernetes then initiates a RollingUpdate, replacing existing pods one at a time (maxUnavailable: 0, maxSurge: 1). This guarantees that at least two replicas remain serving traffic at all times, achieving zero downtime deployment.

5. **Access:** End users and client systems access the hospitality management application through a publicly exposed AWS Elastic Load Balancer (ELB) endpoint automatically provisioned by the Kubernetes LoadBalancer service type. The ELB routes requests to healthy application pods running on the EKS cluster, providing fault-tolerant and scalable access to the application.

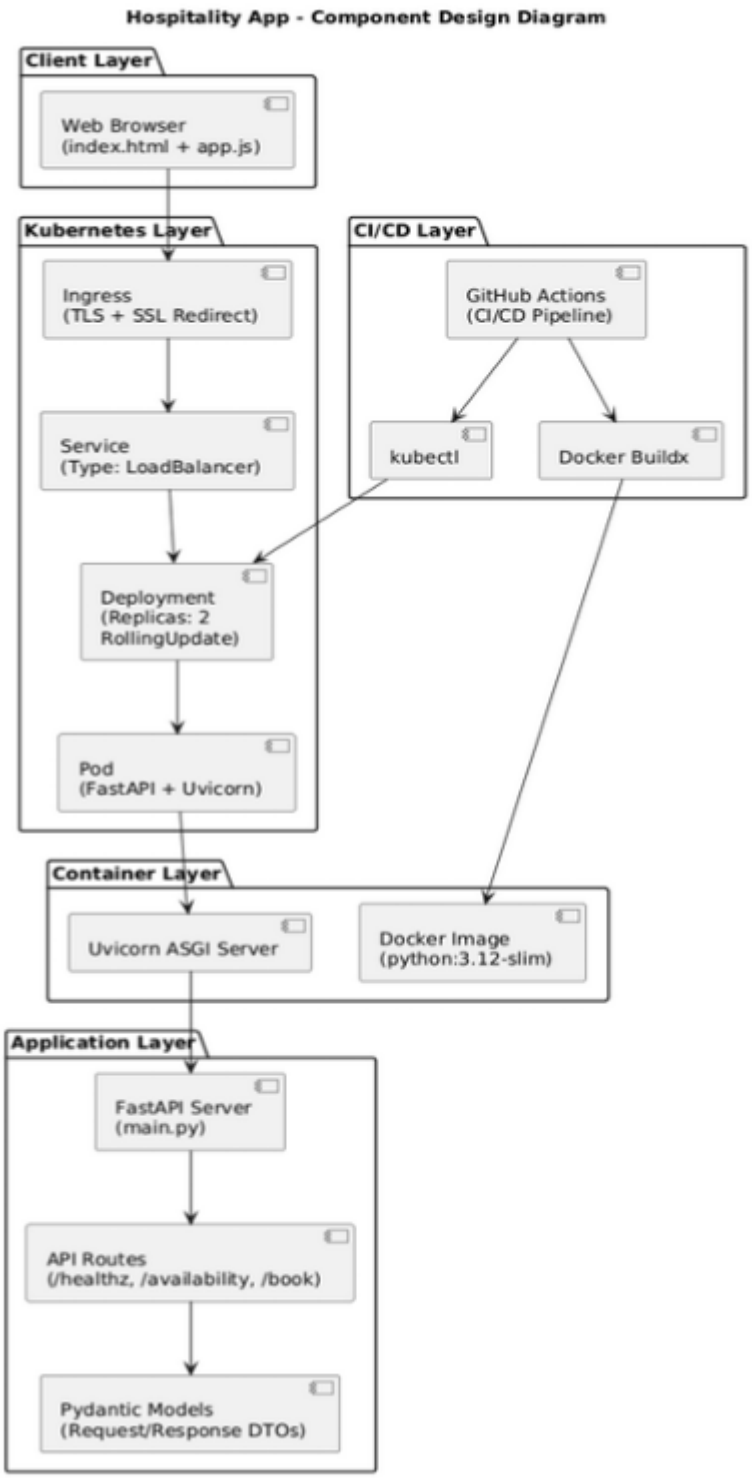
Application Design –



Information Flow –



Component Design –



Technology Stack and Tools –

The project employs a carefully selected set of industry-standard technologies that collectively implement a complete DevOps pipeline. Each tool was chosen based on its maturity, community adoption, and suitability for the specific role it fulfills within the architecture.

Technology / Tool	Role in Project
FastAPI (Python)	Web application framework for the hospitality management REST API
Docker	Application containerization; Dockerfile-based image construction
Docker Hub	Container image registry for storing and distributing versioned images
GitHub	Source code version control and repository hosting
GitHub Actions	CI/CD automation engine; workflow execution on push events
Kubernetes	Container orchestration; manages deployment lifecycle and scaling
AWS EKS	Managed Kubernetes cluster on AWS cloud infrastructure
AWS ELB	Elastic Load Balancer auto-provisioned by Kubernetes LoadBalancer service
Terraform	Infrastructure as Code; AWS EKS cluster provisioning via main.tf
kubectl	Kubernetes CLI used within GitHub Actions to apply deployment updates
YAML	Declarative format for Kubernetes manifests (deployment.yaml, service.yaml)

Frontend –

For the frontend layer, HTML, CSS, and JavaScript are used to design and implement the user interface. HTML provides the structural layout of the web pages, CSS handles styling and visual presentation, and JavaScript enables dynamic behavior and interaction with backend APIs. This combination ensures a lightweight and responsive frontend.

FastAPI Framework -

FastAPI is a modern, high-performance Python web framework built on top of Starlette and Pydantic. It supports asynchronous request handling via Python's asyncio library, enabling high concurrency with low resource usage — particularly valuable for hospitality platforms that may experience burst traffic during booking windows. FastAPI's built-in OpenAPI

documentation generation via Swagger UI and its native support for data validation through Pydantic models make it well-suited for rapid API development. The /healthz endpoint is implemented as a simple GET route returning HTTP 200, enabling Kubernetes health probes to function correctly.

Docker and Containerization -
The application Dockerfile follows a multi-layer build pattern beginning with an official python:3.11-slim base image to minimize attack surface and image size. Application dependencies are installed via pip from a requirements.txt file as a separate layer, leveraging Docker's build cache to accelerate subsequent builds when only application source code changes. The application is launched via uvicorn, the ASGI server, listening on port 8000. The Dockerfile is version-controlled alongside application source code, ensuring the build environment is always synchronized with application requirements.

GitHub Actions CI/CD -
GitHub Actions uses a YAML-defined workflow file (`.github/workflows/deploy.yml`) that specifies the trigger event, execution environment, and sequence of steps. The workflow uses `ubuntu-latest` as the runner environment and leverages official GitHub Actions such as `actions/checkout@v3` for source checkout, `docker/login-action` for Docker Hub authentication using encrypted repository secrets, `docker/build-push-action` for image construction and registry publishing, and AWS credential actions for EKS authentication. The use of `github.sha` as the image tag ensures every deployment references a specific, immutable source code state.

Version Control –
For version control, Git and GitHub are used to track changes and manage collaboration. The project is developed and tested on a Linux operating system, which is widely used in DevOps and production environments.

Results & Output

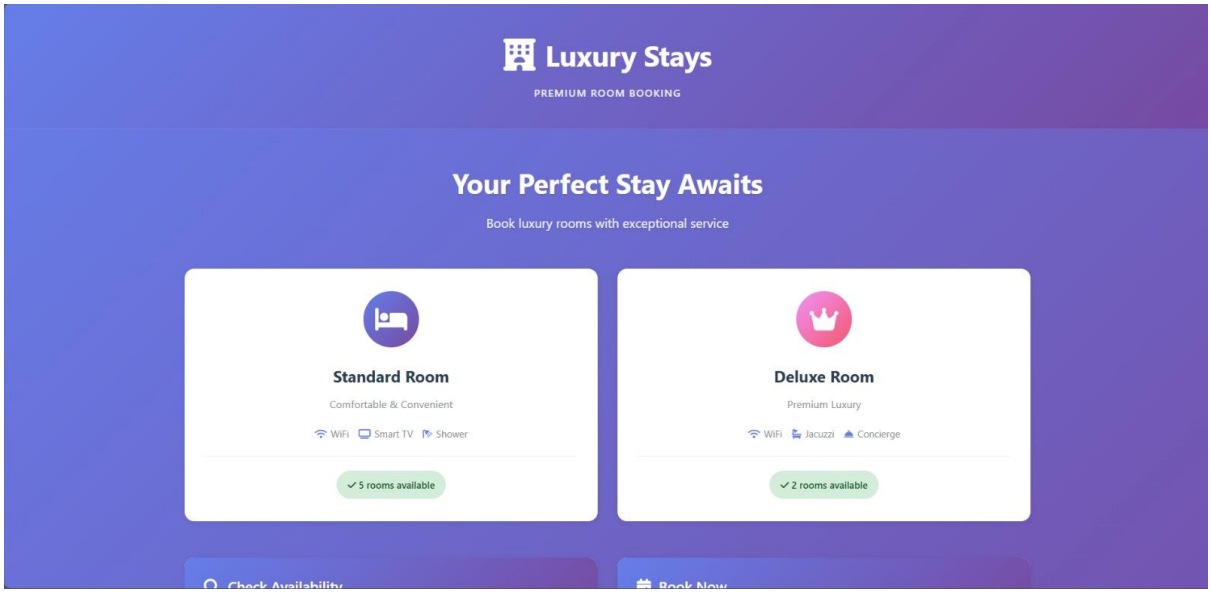
The following outputs were produced and verified during the project execution:

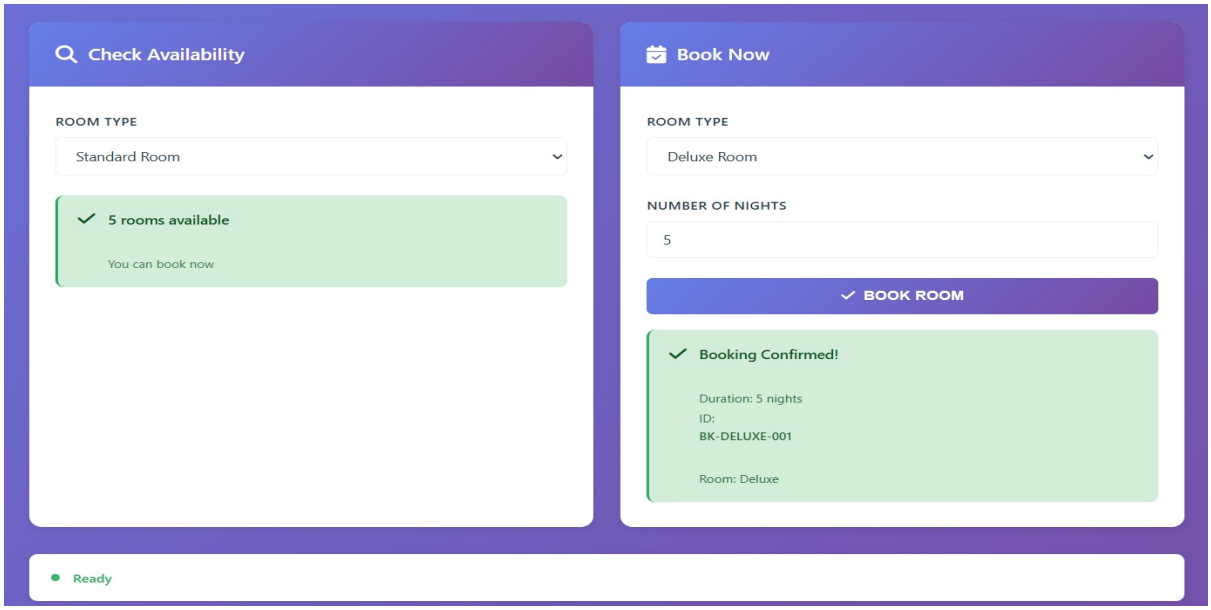
1. CI/CD Pipeline Execution: GitHub Actions workflow successfully triggered on every push to the main branch, producing a Docker image tagged with the Git commit SHA, pushing it to Docker Hub, and updating the Kubernetes deployment without manual intervention.

2. Zero Downtime Deployment Verified: Rolling update was observed via kubectl rollout status, confirming that pod replacement occurred sequentially with no interruption to service. At all times during the update, at least two pods were in Ready state serving live traffic through the LoadBalancer endpoint.
3. Key Outcomes: The hospitality management application was successfully deployed to AWS EKS with two running replicas, a healthy LoadBalancer service with an external IP endpoint, passing /healthz readiness and liveness probes, and a fully automated

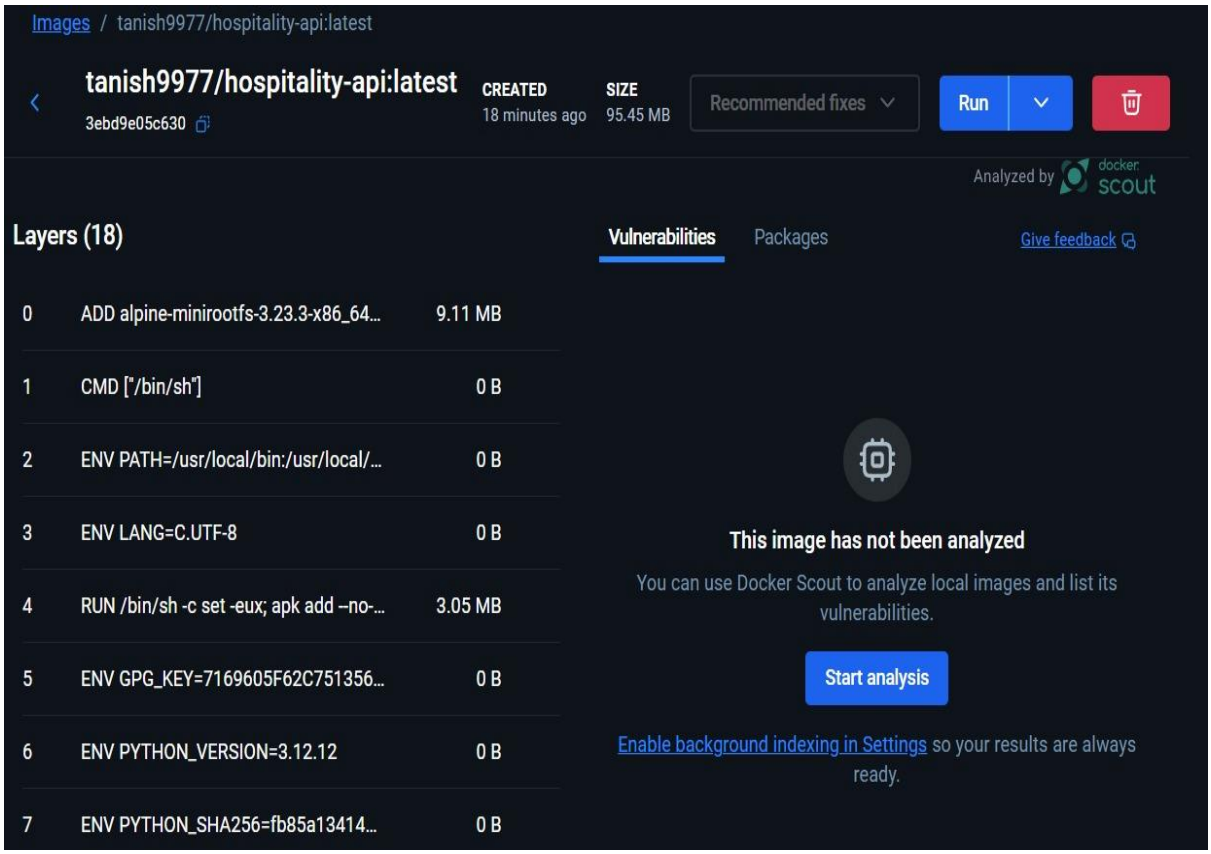
   CI/CD pipeline reducing deployment time from manual hours to under three minutes per release cycle.

Dashboard Page –

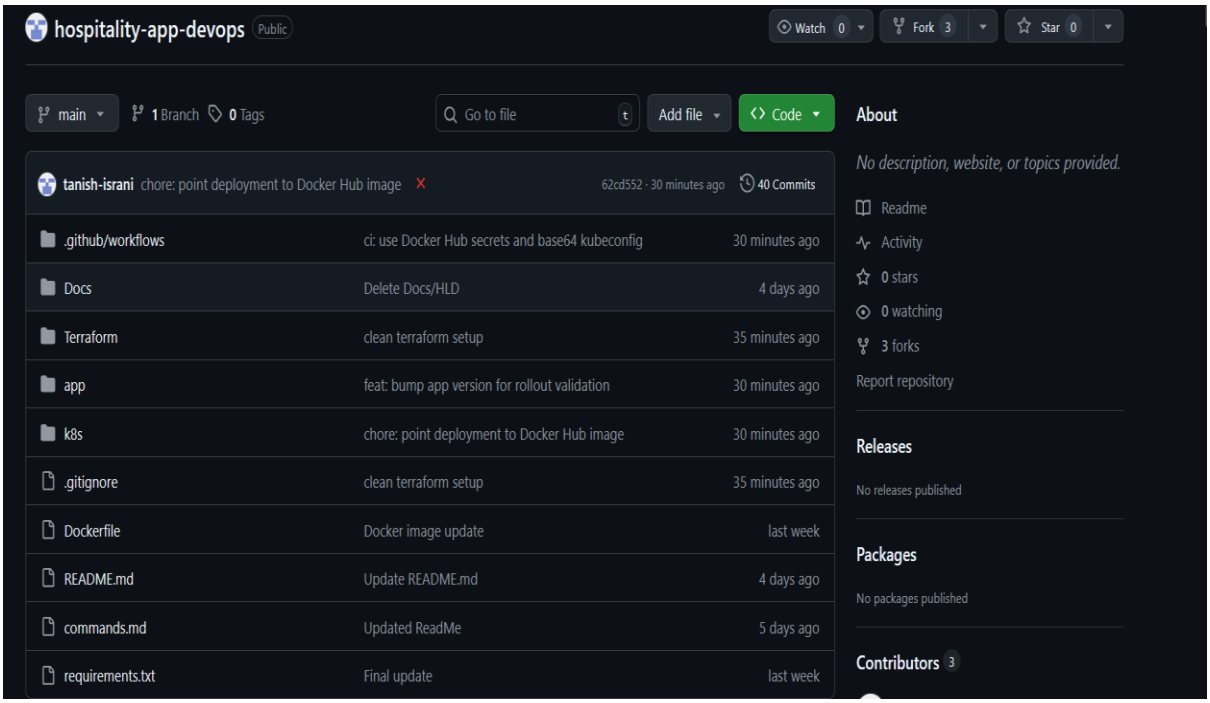




Docker Containerization –

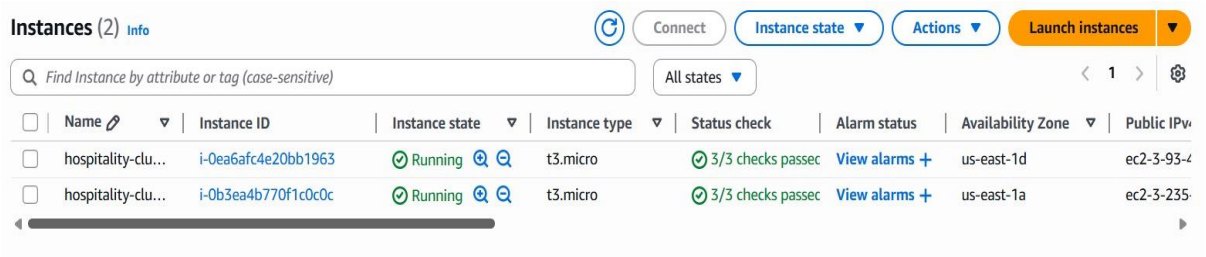


GitHub Repo –

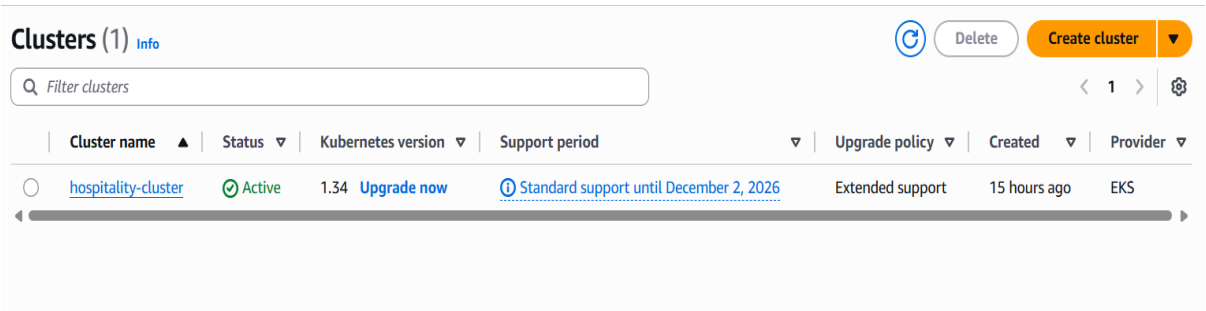

hospitality-app-devops
Public
Watch 0
Fork 3
Star 0
main
1 Branch
0 Tags
Go to file
t
Add file
Code
tanish-israni chore: point deployment to Docker Hub image
62cd552 · 30 minutes ago
40 Commits
.github/workflows
ci: use Docker Hub secrets and base64 kubeconfig
30 minutes ago
Docs
Delete Docs/HLD
4 days ago
Terraform
clean terraform setup
35 minutes ago
app
feat: bump app version for rollout validation
30 minutes ago
k8s
chore: point deployment to Docker Hub image
30 minutes ago
.gitignore
clean terraform setup
35 minutes ago
Dockerfile
Docker image update
last week
README.md
Update README.md
4 days ago
commands.md
Updated ReadMe
5 days ago
requirements.txt
Final update
last week
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
3 forks
Report repository
Releases
No releases published
Packages
No packages published
Contributors 3

Cloud (AWS) –

EC2 Instance –



EKS Cluster –



Key Outcomes -

The Hospitality App DevOps project successfully demonstrated the implementation of a complete end-to-end DevOps workflow for deploying a hospitality management application. A fully functional application environment was established using containerization and cloud-native technologies, ensuring consistent execution across development, testing, and deployment stages. The use of Docker enabled the application and its dependencies to be packaged into portable containers, eliminating environment compatibility issues and improving deployment reliability. Another major outcome of the project was the successful implementation of an automated CI/CD pipeline using GitHub Actions.

The pipeline automatically builds the application, creates Docker images, pushes them to a container registry, and deploys updated versions to the Kubernetes cluster whenever code changes are committed. This automation significantly reduced manual deployment effort, minimized human errors, and enabled faster and repeatable software releases aligned with modern DevOps practices.

The project also achieved scalable and highly available deployment through Kubernetes orchestration and Infrastructure as Code using Terraform. The system supported rolling updates with zero downtime, automated health monitoring using readiness and liveness probes, and cloud infrastructure provisioning through version-controlled configuration files. Overall, the project provided practical experience in DevOps implementation while delivering a reliable, scalable, and production-ready hospitality application deployment architecture.

Conclusion

This project successfully demonstrates the implementation of a production-grade DevOps pipeline for a Hospitality Management Application using modern containerization and orchestration technologies. By integrating GitHub Actions, Docker, Kubernetes, AWS EKS, and Terraform into a unified workflow, the project achieves automated, repeatable, and zero-downtime deployments that align with industry best practices. The RollingUpdate deployment strategy, combined with readiness and liveness probes, ensures continuous service availability even during active release cycles. The stateless application architecture further supports horizontal scalability, making the system capable of handling fluctuating guest traffic without service degradation. Key learnings from this project include hands-on experience with CI/CD pipeline design, Kubernetes workload management, cloud infrastructure provisioning using Infrastructure as Code, and the application of DevOps

principles such as automation, observability, and reliability in a real-world hospitality domain context.

Future Scope & Enhancements

- **Monitoring and Observability Integration:** Implementing Prometheus for metrics collection and Grafana for real-time dashboards to gain deeper visibility into pod performance, resource utilization, request latency, and error rates across the Kubernetes cluster.
- **Security Scanning and DevSecOps Integration:** Integrating security scanning tools such as Trivy for Docker image vulnerability scanning, SonarQube for static code analysis, and Snyk for dependency vulnerability detection directly into the GitHub Actions CI pipeline to enforce security at every stage of the software delivery lifecycle.
- **Horizontal Pod Autoscaling and Serverless Migration:** Enabling Kubernetes Horizontal Pod Autoscaler (HPA) to dynamically scale pod replicas based on CPU and memory metrics, and exploring migration of specific stateless microservices of the hospitality platform to AWS Lambda or Google Cloud Functions to further reduce operational overhead and cost.